Telemetry Packet Evidence

Verify bytes before trusting an observation

A local fixture makes integrity, sequence, and age visible.

Lesson	026 — component	**Time**	35–50 minutes
Prerequisites	Lessons 001–025	**Board**	Arduino Mega 2560
Primary evidence	D45 / TP-LED	**Status**	host verified; physical build unmeasured
Safety boundary	E1, compiled-in fixtures, no radio receiver or transmitter		

1 Mission

Preserve the evidence chain:

$$\text{packet bytes} \rightarrow \text{integrity} \rightarrow \text{accepted observation} \rightarrow \text{local age} \rightarrow \text{visible cadence.}$$

`TelemetryPacketCodec` defines one exact bounded ADK lab packet. `ObservationTracker` makes sequence and local freshness explicit. This packet is not an observed radio protocol. The canonical sketch creates its fixtures locally and neither receives nor emits RF.

> **Integrity is not trust.** CRC-16 detects accidental changes. It does not authenticate a sender, hide a payload, prove provenance, prevent replay, or make an unknown capture lawful to retain.

1.1 Learning outcomes

- distinguish packet, sample, observation, and record;
- explain exact-length decoding and unchanged output on failure;
- classify first, in-order, duplicate, gap, and reordered sequence values;
- calculate freshness from local receipt time rather than a remote clock;
- separate transport corruption, sample quality, and staleness;
- prove acquisition and safe state with separate circuit evidence.

2 Parts, energy, and wiring

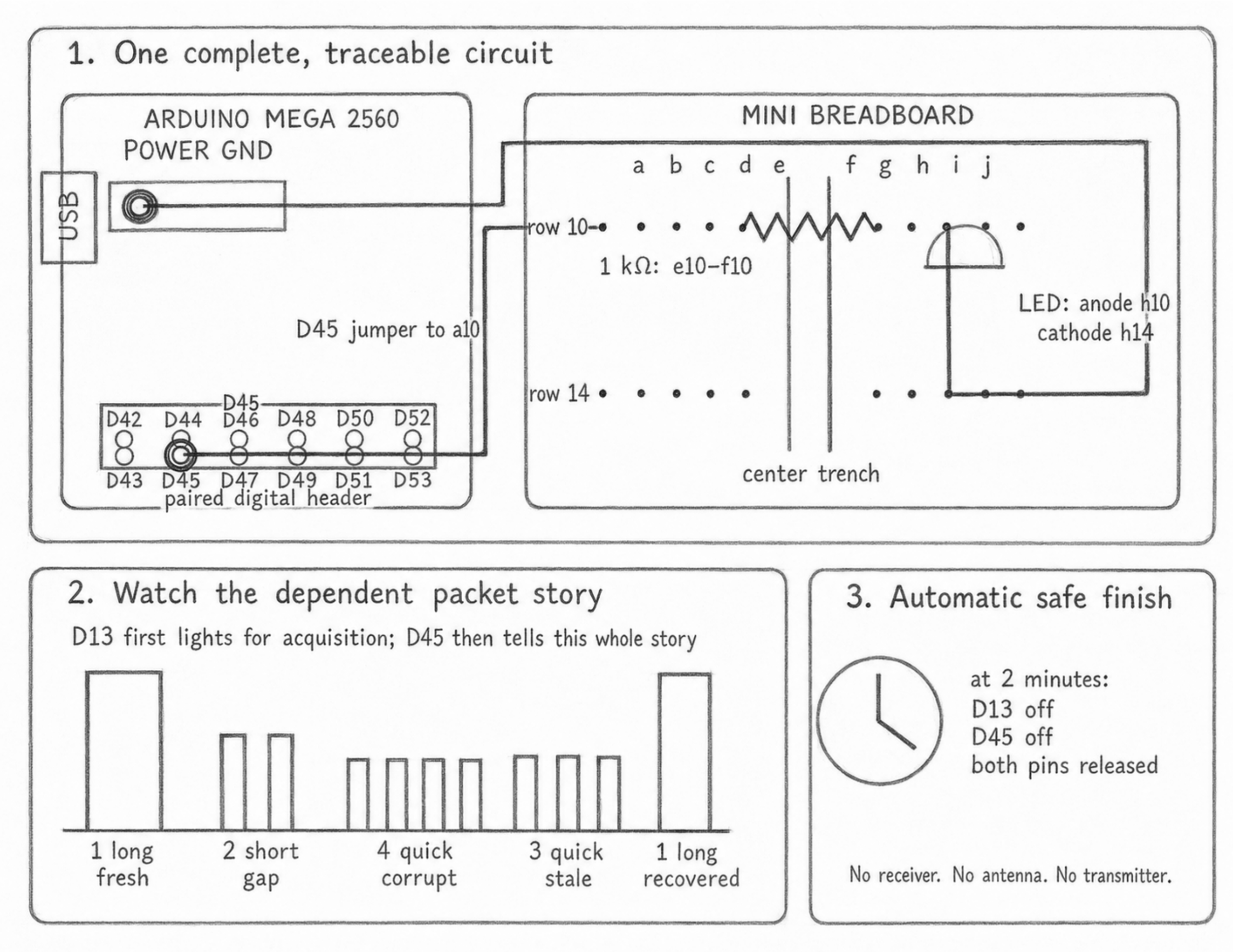
Part	Requirement
Arduino Mega 2560	USB-powered board
Evidence LED	One ordinary LED and one 1 kΩ resistor
Instrument	Logic probe or oscilloscope; meter for static safe-state check
Fixtures	Compiled-in ADK-owned bytes; no external receiver

For an illustrative 5 V rail and 2 V LED drop,

$$I = \frac{5\ \text{V} - 2\ \text{V}}{1000\ \Omega} = 3\ \text{mA}.$$

Measure the actual rail, resistor, and LED drop before recording current. Remove USB power before changing wires or probe clips.

Signal	Mega pin	Connection
Acquisition evidence	D13	built-in LED
Observation evidence / TP-LED	D45	D45 → 1 kΩ → LED anode; cathode → GND
Probe reference	GND	common reference for TP-LED



Alternative text: three-panel graphite wiring plate identifying the Mega's D45 and GND header holes, breadboard columns and rows, a one-kilohm series resistor, and LED anode and cathode. Later panels show the one-, two-, four-, and three-blink packet story. No radio part is present.

2.1 Build, upload, watch

With USB unplugged, connect D45 to one end of the 1 kΩ resistor. Use a10 for the D45 jumper, place the resistor from e10 to f10 across the center trench, put the LED anode (long leg) at h10 and cathode (short leg, flat rim) at h14, then join h14 directly to Mega GND. Do not put both resistor leads or both LED legs in one connected five-hole strip.

Upload once and watch the complete 12-second packet story:

1. D13 lights when the software and both output pins are acquired.
2. One long D45 blink announces fresh, in-order evidence.
3. Two short blinks expose the deliberately skipped sequence.
4. Four quick blinks catch a deliberately corrupted packet.
5. Three quick blinks show the last accepted packet becoming stale.

6. One long blink returns with the recovered fresh packet.

The story repeats until the fixed two-minute deadline, when the sketch switches both outputs off and releases their pins. Every later scene depends on the same correct wiring and preceding packet path; there is no separate learner test.

3 Exact bounded packet

The wire image is 19 bytes. It contains no pointer, padding, native enumeration, native Boolean, or compiler-dependent structure image. Multibyte fields are big-endian; signed fields use two's-complement bits.

Offset	Bytes	Field	Meaning
0	1	magic	`0xAD`
1	1	version	`0x01`
2	2	source ID	opaque lab identifier
4	2	sequence	ordering evidence
6	4	observed ms	untrusted remote-clock evidence
10	1	kind	bounded telemetry kind
11	1	quality	bounded sample quality
12	4	value	signed integer
16	1	exponent	signed decimal scale
17	2	CRC-16	accidental-corruption check

CRC-16/CCITT-FALSE uses polynomial `0x1021`, initial value `0xFFFF`, no reflection, no final XOR, and covers bytes 0–16. It is not a security primitive.

`encode()` returns `Result<uint16_t>`; callers check `ok()` before reading `value()`. A short output span receives no partial packet. `decode()` requires exactly 19 bytes and distinguishes bad version, length, type, quality, integrity, and trailing data. Failure leaves the destination unchanged.

A changed value or timestamp bit with the old CRC reaches `BadIntegrity`. A changed magic, version, kind, or quality may reach its semantic validator first. Tests recompute CRC after a deliberate semantic change when isolating `BadVersion`, `BadType`, or `BadQuality`. Every one-bit mutation is rejected; the exact class follows this precedence.

4 Sequence is evidence, not arrival order

The tracker owns one source ID and remembers one accepted observation. `First` establishes the baseline. The next sequence is `InOrder`; a larger forward jump is `Gap`. Equal is `Duplicate`; evidence behind the accepted value is `Reordered`. Comparison uses a documented unsigned half-range rule so wrap from 65535 to 0 remains deterministic.

The exact delta `0x8000` is `Reordered`; it never replaces the accepted observation.

Duplicate and reordered packets do not replace the accepted sample. A forward gap may replace it, but the gap remains visible. Source mismatch is invalid input rather than an observation for a different tracker.

Accepted	Incoming	State	Does value change?
—	100	First	yes
100	101	InOrder	yes
101	101	Duplicate	no
101	104	Gap	yes, with gap evidence
104	103	Reordered	no
65535	0	InOrder	yes, across wrap

5 Freshness belongs to the receiver

Age is `now - receivedAt`, not `now - observedMilliseconds`. The remote timestamp is retained for diagnosis, but a future, reversed, or drifting remote clock cannot keep a packet fresh.

The example uses:

$$\text{Fresh} < 2000 \text{ ms}, \qquad 2000 \leq \text{Aging} < 5000 \text{ ms}, \qquad \text{Stale} \geq 5000 \text{ ms}.$$

Local age	State	Visible consequence
1999 ms	Fresh	one long pulse
2000 ms	Aging	two short pulses
4999 ms	Aging	two short pulses
5000 ms	Stale	three quick pulses

Reversing the packet's remote timestamp does not alter these results because freshness follows local receipt time.

6 Narrative run loop and visible state

```
observePacket           (now);
verifyPacket            (now);
updateFreshness         (now);
showObservationEvidence (now);
```

Objects appear in dependency order: runtime, acquisition LED, packet-evidence LED, pure codec, then tracker. Setup acquires, configures, and starts. The loop observes one scheduled fixture, verifies exact packet bytes, updates freshness, and actuates D45.

The 12-second schedule accepts sequence 1, skips 2 and accepts 3, corrupts the next packet, waits until sequence 3 becomes stale, then accepts sequence 4 when the schedule restarts. Every advertised evidence state is reachable, and recovery is an accepted in-order observation rather than a display reset.

`TelemetryFixtureSchedule` owns this tested one-shot decision trace; the sketch does not duplicate its timing policy. `TelemetryEvidenceModel` applies the tested precedence fault, corrupt, stale, sequence/aging, then fresh. The four run-loop calls therefore name actual work rather than forwarding into another private state machine. An unexpected execution fault uses the safe-off path; the four-pulse fixture demonstrates corrupt packet evidence, not a claim that a failed output can diagnose itself.

State	D45 cadence	Meaning
Fresh, in order	one 1 s pulse	accepted observation is fresh
Sequence or aging	two 150 ms pulses	duplicate, gap, reorder, or aging
Stale	three 120 ms pulses	no recent accepted observation
Corrupt fixture	four 100 ms pulses	packet integrity failed

D13 proves acquisition only. D45 reports current evidence only. Serial may print supporting records, but it does not control tuning, capture, or replay and is never the sole observation.

7 Read the adventure

D13 first tells you acquisition succeeded. D45 then tells the packet story. If D13 lights but D45 stays dark, check the D45 header hole, resistor row, LED polarity, and GND lead. If D45 blinks but never changes cadence, the physical path is probably intact and the fixture/state path needs review. A complete one–two–four–three–one sequence demonstrates the local software chain; it proves no radio, network, remote sensor, or security property.

After two minutes both LEDs go dark by design. A dark LED is useful visible progress, but only a separately measured TP-LED voltage and pin state could establish physical safe-state evidence. This lesson makes no such unrecorded claim.

8 Diagnosis and privacy review

Observation	Check next
D13 dark	tracker configuration, output claim conflict, board power
D13 on; D45 dark	fixture schedule, LED polarity, resistor, TP-LED
valid becomes corrupt	exact length, version, byte order, CRC coverage
gap never appears	sequence fixture and half-range comparison
stale never appears	local receipt time and exact 5 s boundary
state changes from remote clock	defect: freshness must use local receipt
D45 driven after stop	safe-state failure; remove USB and record deviation

Use only owned lab fixtures. Do not collect device serial numbers, addresses, locations, credentials, or unnecessary payloads. This lesson contains no receiver, transmitter, raw capture export, rolling-code analysis, key recovery, replay, or command mapping.

9 What is verified behind the scenes

Automated tests retain golden packet bytes, every one-bit corruption, truncation, extension, short output, all sequence classes, wraparound, freshness boundaries, remote-clock independence, lifecycle, and deterministic replay. The Mega build and size measurement are automated too. Those results verify software behavior, not this particular board, LED, resistor, wiring, or probe measurement.

10 Further reading

- HTML reference and corrections
- TelemetryPacketCodec public header and implementation

- packet golden-vector and corruption tests
- ObservationTracker public header and implementation
- tracker lifecycle and timing tests
- fixture schedule and evidence model and fixture and visible-precedence tests
- Lessons 025–027 executable design
- Arduino Mega 2560 Rev3 datasheet